ParaRNN

Параллельное обучение нелинейных RNN без линеаризации скрытого состояния

Danieli et al. · Apple · ICLR 2026 · arXiv:2510.21450

Efficient Neural Networks · Университет ИТМО · ДЗ-2

Последовательная развёртка RNN

На этапе вывода классические LSTM и GRU привлекательны компактным скрытым состоянием $O(1)$, константным временем генерации токена и отсутствием KV-кэша. Основной барьер возникает при обучении: пошаговая зависимость (1) задаёт временную глубину $O(L)$. Вычисление h_l строго требует результата h_{l-1}, что препятствует эффективному распараллеливанию на GPU.

$$h_l = f(h_{l-1}, x_l), \qquad l = 1, \ldots, L, \quad h_0 = 0 \tag{1}$$

В отличие от подходов, переходящих к механизму внимания или упрощающих рекуррентность до линейной (как в SSM), ParaRNN сохраняет выразительную нелинейную функцию f. Вместо изменения архитектуры ячейки здесь распараллеливается само численное нахождение всей траектории H во времени.

Линейные SSM и associative scan

Чтобы преодолеть барьер $O(L)$ при обучении, современные State Space Models (SSM) упрощают переход между шагами до строго линейного:

$$h_l = A_l h_{l-1} + B_l x_l \tag{2}$$

Линейные отображения ассоциативны относительно композиции. Это позволяет заменить последовательную развёртку алгоритмом parallel scan (prefix sum), сократив вычислительную глубину до логарифмической:

	Последовательный проход	Parallel scan (SSM)
Вычислительная глубина	$O(L)$	$O(\log L)$
Суммарный объём работы	$O(L)$	$O(L)$
Требование к шагу перехода	Произвольный (f)	Строгая ассоциативность (линейность)

Цена линейности

Линейность по состоянию — вынужденная мера ради параллелизации на GPU. Платой за неё становится потеря выразительности: теоретически показано, что чисто линейные рекуррентности не способны моделировать сложные пошаговые зависимости и задачи с отслеживанием состояния (например, подсчёт чётности / *parity*).

Задача ParaRNN — вернуть скрытому состоянию нелинейность, сохранив параллельное обучение.

Развёртка как $F(H) = 0$

Вместо пошагового вычисления по времени вся траектория скрытых состояний $H = [h_1, \dots, h_L]^\top$ рассматривается как корень единой системы из L нелинейных уравнений:

$$F_l(H) = h_l - f(h_{l-1}, x_l) = 0, \qquad l = 1, \dots, L \tag{3}$$

где граничное условие $h_0 = 0$ делает систему замкнутой и однозначно разрешимой (L уравнений для L неизвестных).

Суть подхода авторов:

- **Итерации метода Ньютона.** Вместо последовательного цикла по времени система решается численно — через последовательную линеаризацию невязок $F(H)$.
- **Марковская структура.** Поскольку шаг l зависит только от текущего h_l и предыдущего h_{l-1}, глобальный якобиан системы получается блочно-бидиагональным. Это позволяет не хранить плотную матрицу размера $(L \cdot d_h)^2$.
- **Параллельная инициализация.** Стартовая точка $h_l^{(0)} = f(0, x_l)$ вычисляется независимо для всех токенов за один параллельный проход по последовательности.

Ньютон и parallel scan

На каждой итерации k система невязок линеаризуется вокруг текущей траектории $H^{(k)}$:

$$J_F(H^{(k)})\,\Delta H = -F(H^{(k)}), \qquad H^{(k+1)} = H^{(k)} + \Delta H \tag{4}$$

Поскольку глобальный якобиан J_F блочно-бидиагонален, строчка l матричной системы раскрывается в локальное уравнение для поправок:

$$\Delta h_l = J_f\big|_{h_{l-1}}\,\Delta h_{l-1} + r_l, \qquad r_l = f(h_{l-1}, x_l) - h_l \tag{5}$$

Главный инсайт авторов. Уравнение поправок (5) — линейная рекуррентность:

- локальный якобиан J_f играет роль матрицы перехода (A_l);
- невязка r_l играет роль входного вектора ($B_l x_l$).

Линейная зависимость ассоциативна, поэтому вектор поправок ΔH находится не последовательным проходом, а алгоритмом parallel scan за $O(\log L)$.

Итоговая схема: внешний решатель (метод Ньютона) выполняет несколько итераций уточнения, а внутренний решатель на каждом шаге распараллеливается по всей длине последовательности на GPU.

Сходимость метода Ньютона

Условие выигрыша по времени

Параллельный солвер быстрее последовательного прохода $O(L)$, пока выполняется условие:

$$N_{\text{its}} \ll \frac{L}{\log_2 L}$$

Это даёт большой запас: даже при сублинейном числе итераций (например, $N_{\text{its}} \sim O(\log L)$) глубина $\mathcal{O}(\log^2 L)$ асимптотически обгоняет последовательную развёртку.

Эмпирическая сходимость в статье (рис. 4)

На практике этот запас почти не расходуется — невязка затухает практически мгновенно:

- максимальная невязка $\|F\|_\infty$ падает до машинной точности (10^{-5}–10^{-6}) за 3 шага;
- скорость сходимости инвариантна к длине: кривые для $L = 2^8$ и $L = 2^{11}$ накладываются друг на друга;
- поведение одинаково устойчиво на случайных весах и на обученных моделях до 7B.

Для обучения фиксируется константный бюджет $N_{\text{its}} = 3$.

Оговорка

Сходимость за 3 шага обеспечена диагональной структурой ячеек и сжатием весов ($|a_*| \leq 0{,}5$). Для произвольных новых RNN поведение солвера потребует отдельной валидации.

Диагонализация матриц перехода: путь к $\mathcal{O}(d_h)$

В чём проблема исходных ячеек

В стандартных LSTM и GRU скрытые состояния смешиваются плотными матрицами. В параллельном скане это потребовало бы перемножать блоки $d_h \times d_h$ вдоль всей цепочки — схема становится непрактичной: $\mathcal{O}(Ld_h^2)$ памяти и $\mathcal{O}(d_h^3)$ операций на каждый шаг редукции.

Решение: диагональные веса ячейки

Авторы делают диагональными сами параметры перехода и peephole-связи:

$$A_* = \mathrm{diag}(a_*), \qquad C_* = \mathrm{diag}(c_*), \qquad a_*, c_* \in \mathbb{R}^{d_h} \tag{6}$$

Это даёт строго диагональный якобиан у ParaGRU и блочно-диагональный 2×2 у ParaLSTM (из-за пары состояний c_l и h_l), уменьшая расходы памяти с $\mathcal{O}(Ld_h^2)$ до $\mathcal{O}(Ld_h)$ и критически снижая сложность вычислений с $\mathcal{O}(d_h^3)$ до $\mathcal{O}(d_h)$.

Уязвимость в аргументации авторов

- Диагонализация превращает d_h-мерную ячейку в d_h изолированных одномерных рекуррентностей — каналы во времени больше не взаимодействуют напрямую.
- Авторы утверждают, что смешивание каналов компенсируют внешние слои MLP. Но выразительность они исследовали ограниченно: на синтетических задачах проверена лишь простая $\mathbb{Z}_2$-Parity, для которой межканальное взаимодействие не нужно (достаточно одного скалярного регистра).
- Вопрос о том, способна ли связка «1D-нелинейность + послойный MLP» заменить полноценную многомерную нелинейную динамику, в статье остаётся открытым.

Обратный проход: один скан вместо метода Ньютона

Дифференцирование рекуррентности (BPTT) даёт строго линейную цепочку по градиентам скрытого состояния:

$$\nabla_{h_{l-1}}\mathcal{L} = \left(J_f|_{h_{l-1}}\right)^\top \nabla_{h_l}\mathcal{L} + \partial_{h_{l-1}}\mathcal{L}, \qquad l = L, \ldots, 1 \tag{7}$$

с граничным условием $\nabla_{h_L}\mathcal{L} = \partial_{h_L}\mathcal{L}$.

Поскольку система линейна, итерации метода Ньютона не требуются, и все градиенты $\nabla_h\mathcal{L}$ находятся за один обратный parallel scan.

Бюджет вычислений на полный шаг обучения (Forward + Backward):

- Mamba: 1 (fwd) + 1 (bwd) = 2 скана
- ParaRNN: 3 (fwd, метод Ньютона) + 1 (bwd) = 4 скана

Суммарные накладные расходы нелинейности на шаге обучения составляют всего $\sim 2\times$ относительно чисто линейных SSM.

Реализация на GPU: аппаратный parallel scan

Чтобы обеспечить вычислительную эффективность, авторы разработали собственное CUDA-ядро, раскладывающее параллельную редукцию по всей аппаратной иерархии GPU:

Уровень	Механика
Thread (registers)	Локальный chunk 2–4 уравнений (`chunk_size`), прямое исключение
Warp (registers)	Обмен через `__shfl_down_sync`, без обращения к памяти
Block (Shared memory)	Сведение результатов варпов с обратной подстановкой
Grid (Global memory)	При $L \ge 2^{16}$ — сетка блоков с синхронизацией через VRAM

Уровни интеграции в ParaRNN

- **Modular (PyTorch + CUDA):** сборка якобиана выполняется в PyTorch (через autograd или аналитически), сам скан — в CUDA. Это сохраняет гибкость при создании новых типов ячеек.
- **Fully-fused CUDA:** весь цикл метода Ньютона (невязки, сборка якобиана и скан) объединён в одно ядро. Это устраняет накладные расходы на промежуточный обмен данными через VRAM.

Производительность (A100, $L = 2^9$)

Ускорение до $\sim 665\times$ относительно последовательной развёртки ячейки. В режиме Fully-fused время исполнения ячейки в $1{,}5\times$ (ParaLSTM) – $2{,}6\times$ (ParaGRU) меньше, чем у Mamba.

Экспериментальные результаты

Авторы валидируют ParaRNN по двум направлениям: выразительность рекуррентной ячейки на синтетических тестах и качество при масштабировании языковой модели до 7B параметров.

Модель	MQAR	k-hop	Parity	PPL↓ (7B)
Transformer	100%	78%	~53%	9,55
Mamba2	100%	~98%	~51%	**8,62**
ParaLSTM	**100%**	**100%**	**100%**	9,16
ParaGRU	**100%**	**100%**	**100%**	9,19

Синтетические тесты (1 слой) · SlimPajama 7B

Ключевые выводы

- **Выразительность нелинейности.** На задачах отслеживания состояния (k-hop) и периодической логики (Parity) чисто линейная Mamba2 падает до уровня случайного угадывания (~51%), тогда как ParaGRU и ParaLSTM решают их на 100%.
- **Масштабирование до 7B.** При обучении по законам масштабирования Chinchilla нелинейные RNN обходят baseline Transformer по перплексии (9,16–9,19 vs 9,55) и показывают устойчивую сходимость градиентов без деградации качества.

Архитектура и код

Для тестов авторы адаптировали трансформерный стек DCLM: механизм внимания заменили на блок ParaRNN (ParaGRU/ParaLSTM) с каузальной свёрткой и гейтированием в стиле Mamba, оставив перемешивание каналов внешним слоям MLP. Модели от 400M до 7B параметров обучались на SlimPajama (без Books3) по законам масштабирования Chinchilla.

Фреймворк `apple/ml-pararnn` автоматизирует параллелизацию: достаточно описать шаг f и аналитический якобиан J_f, а сборку уравнений и параллельный скан библиотека выполняет сама. На этапе вывода модель переходит в стандартный рекуррентный режим без итераций метода Ньютона, сохраняя преимущество классических RNN — работу с памятью $O(1)$ без KV-кэша.

Воспроизведение

Для воспроизведения статьи разработана библиотека `pararnn-torch` (`github.com/bugkira/pararnn-torch`). На обучении она считает всю траекторию скрытых состояний методом Ньютона со сканом; в режиме вывода остаётся тот же переход f, но уже по одному шагу за раз. Согласие параллельного и последовательного проходов проверяется штатным вызовом API этой же библиотеки.

В численных экспериментах статьи заданы такие параметры:

Параметр	В статье
Init	$h_l^{(0)} = f(0, x_l)$
Ньютон	$N_{\text{its}} = 3$
$\lvert a_* \rvert$	≤ 0,5 на текстовых данных, ≤ 0,9 на синтетических задачах

Выводы

Нелинейную развёртку RNN можно эффективно распараллелить, если искать всю траекторию состояний сразу как корень системы $F(H) = 0$ методом Ньютона. Ключевое инженерное условие такой схемы — диагонализация матриц перехода A_*: она меняет асимптотику сложности одного шага скана с кубической $\mathcal{O}(d_h^3)$ на линейную $\mathcal{O}(d_h)$ и делает вычисления на GPU сопоставимыми по стоимости с линейными SSM.

На практике для ParaGRU и ParaLSTM уже трёх шагов метода Ньютона достаточно, чтобы параллельное приближение совпало с точной пошаговой развёрткой. В результате классические рекуррентные ячейки масштабируются до 7B без потери качества на языковом моделировании, но сохраняют нелинейную динамику там, где линейные модели систематически не справляются — например, на задаче Parity.

`pararnn-torch` · `apple/ml-pararnn` · arXiv:2510.21450
Danieli et al., ICLR 2026 · ИТМО · Efficient NN · ДЗ-2